

```
SELECT datname FROM pg_database;
```

```
CREATE TABLE spotify_staging (  
  id INTEGER,  
  track_id VARCHAR(50),  
  artists TEXT,  
  album_name TEXT,  
  track_name TEXT,  
  popularity INTEGER,  
  duration_ms INTEGER,  
  explicit BOOLEAN,  
  danceability NUMERIC(5,4),  
  energy NUMERIC(5,4),  
  key INTEGER,  
  loudness NUMERIC(6,2),  
  mode INTEGER,  
  speechiness NUMERIC(5,4),  
  acousticness NUMERIC(6,5),  
  instrumentalness NUMERIC(8,7),  
  liveness NUMERIC(5,4),  
  valence NUMERIC(5,4),  
  tempo NUMERIC(7,3),  
  time_signature INTEGER,  
  track_genre VARCHAR(100)  
);
```

```
select * from spotify_staging;
```

```
COPY spotify_staging (  
  id, track_id, artists, album_name, track_name, popularity,  
  duration_ms, explicit, danceability, energy, key, loudness,  
  mode, speechiness, acousticness, instrumentalness, liveness,  
  valence, tempo, time_signature, track_genre  
)  
FROM 'S:\dataset\dataset.csv'  
WITH (FORMAT csv, HEADER true, DELIMITER ',');
```

```
-- DATA FAMILIARITY
```

```
-- Sample Records
```

```
SELECT * FROM spotify_staging LIMIT 10;
```

```
-- Total Records
```

```
SELECT COUNT(*) AS total_records FROM spotify_staging;
```

-- Table Structure

```
SELECT
    column_name,
    data_type,
    is_nullable
FROM information_schema.columns
WHERE table_name = 'spotify_staging'
ORDER BY ordinal_position;
```

-- Unique Counts

```
SELECT
    COUNT(DISTINCT artists) AS unique_artists,
    COUNT(DISTINCT album_name) AS unique_albums,
    COUNT(DISTINCT track_genre) AS total_genres
FROM spotify_staging;
```

-- DATA QUALITY

-- Missing Values

```
SELECT
    COUNT(*) AS total_rows,
    SUM(CASE WHEN track_id IS NULL THEN 1 ELSE 0 END) AS null_track_id,
    SUM(CASE WHEN artists IS NULL OR TRIM(artists) = " THEN 1 ELSE 0 END) AS
null_artists,
    SUM(CASE WHEN track_name IS NULL OR TRIM(track_name) = " THEN 1 ELSE 0
END) AS null_track_name,
    SUM(CASE WHEN popularity IS NULL THEN 1 ELSE 0 END) AS null_popularity,
    SUM(CASE WHEN track_genre IS NULL OR TRIM(track_genre) = " THEN 1 ELSE 0
END) AS null_genre
FROM spotify_staging;
```

-- Duplicate Tracks

```
SELECT
    track_name,
    artists,
    COUNT(*) AS duplicate_count
FROM spotify_staging
GROUP BY track_name, artists
HAVING COUNT(*) > 1
ORDER BY duplicate_count DESC
LIMIT 20;
```

-- Data Validation Issues

```
SELECT 'Invalid Popularity' AS issue, COUNT(*) FROM spotify_staging WHERE popularity
< 0 OR popularity > 100
UNION ALL
```

```
SELECT 'Invalid Duration', COUNT(*) FROM spotify_staging WHERE duration_ms <= 0
UNION ALL
SELECT 'Missing Genre', COUNT(*) FROM spotify_staging WHERE track_genre IS NULL
OR TRIM(track_genre) = '';
```

-- 6. EXPLORATORY DATA ANALYSIS

-- Top 10 Most Popular Songs

```
SELECT track_name, artists, album_name, popularity
FROM spotify_staging
ORDER BY popularity DESC LIMIT 10;
```

-- Top 10 Artists

```
SELECT artists, COUNT(*) AS total_tracks
FROM spotify_staging
GROUP BY artists
ORDER BY total_tracks DESC LIMIT 10;
```

-- Genre Distribution

```
SELECT track_genre, COUNT(*) AS total_tracks
FROM spotify_staging
GROUP BY track_genre
ORDER BY total_tracks DESC;
```

-- Average Popularity by Genre

```
SELECT
    track_genre,
    ROUND(AVG(popularity), 2) AS avg_popularity,
    COUNT(*) AS total_tracks
FROM spotify_staging
GROUP BY track_genre
ORDER BY avg_popularity DESC;
```

-- Average Duration by Genre

```
SELECT
    track_genre,
    ROUND(AVG(duration_ms)/60000.0, 2) AS avg_duration_minutes
FROM spotify_staging
GROUP BY track_genre
ORDER BY avg_duration_minutes DESC;
```

-- Popularity Distribution

```
SELECT
    CASE
        WHEN popularity >= 80 THEN 'Very Popular'
        WHEN popularity >= 60 THEN 'Popular'
```

```

        WHEN popularity >= 40 THEN 'Average'
        WHEN popularity >= 20 THEN 'Less Popular'
        ELSE 'Low Popularity'
    END AS popularity_category,
    COUNT(*) AS total_tracks
FROM spotify_staging
GROUP BY popularity_category
ORDER BY total_tracks DESC;

```

-- Audio Features by Genre

```

SELECT
    track_genre,
    ROUND(AVG(danceability), 3) AS avg_danceability,
    ROUND(AVG(energy), 3) AS avg_energy,
    ROUND(AVG(valence), 3) AS avg_valence,
    ROUND(AVG(tempo), 2) AS avg_tempo
FROM spotify_staging
GROUP BY track_genre
ORDER BY avg_energy DESC;

```

-- BUSINESS INSIGHTS

-- Top Artists by Average Popularity

```

SELECT
    artists,
    ROUND(AVG(popularity), 2) AS avg_popularity,
    COUNT(*) AS total_tracks
FROM spotify_staging
GROUP BY artists
HAVING COUNT(*) >= 5
ORDER BY avg_popularity DESC
LIMIT 10;

```

-- Most Productive Artists

```

SELECT
    artists,
    COUNT(DISTINCT album_name) AS total_albums,
    COUNT(*) AS total_tracks
FROM spotify_staging
GROUP BY artists
ORDER BY total_tracks DESC
LIMIT 10;

```

-- Top Artist in Each Genre

```

SELECT *
FROM (

```

```

SELECT
    track_genre,
    artists,
    COUNT(*) AS total_tracks,
    ROW_NUMBER() OVER (PARTITION BY track_genre ORDER BY COUNT(*) DESC)
AS rn
FROM spotify_staging
GROUP BY track_genre, artists
) ranked
WHERE rn = 1
ORDER BY track_genre;

```

-- Explicit vs Non-Explicit

```

SELECT
    explicit,
    COUNT(*) AS total_tracks,
    ROUND(AVG(popularity), 2) AS avg_popularity
FROM spotify_staging
GROUP BY explicit
ORDER BY avg_popularity DESC;

```

-- Duration Category vs Popularity

```

SELECT
    CASE
        WHEN duration_ms < 180000 THEN 'Short (<3 min)'
        WHEN duration_ms BETWEEN 180000 AND 300000 THEN 'Medium (3-5 min)'
        ELSE 'Long (>5 min)'
    END AS duration_category,
    ROUND(AVG(popularity), 2) AS avg_popularity,
    COUNT(*) AS total_tracks
FROM spotify_staging
GROUP BY duration_category
ORDER BY avg_popularity DESC;

```

-- ADVANCED ANALYSIS

-- Most Consistent Artists

```

SELECT
    artists,
    COUNT(*) AS total_tracks,
    ROUND(AVG(popularity), 2) AS avg_popularity,
    ROUND(STDDEV(popularity), 2) AS popularity_variation
FROM spotify_staging
GROUP BY artists
HAVING COUNT(*) >= 10
ORDER BY popularity_variation ASC

```

```
LIMIT 10;
```

```
-- Artists with Multiple Genres
```

```
SELECT
    artists,
    COUNT(DISTINCT track_genre) AS genres_covered
FROM spotify_staging
GROUP BY artists
HAVING COUNT(DISTINCT track_genre) > 1
ORDER BY genres_covered DESC
LIMIT 15;
```

```
-- Top Genres in High Popularity Songs
```

```
SELECT
    track_genre,
    COUNT(*) AS total_hits
FROM spotify_staging
WHERE popularity >= 80
GROUP BY track_genre
ORDER BY total_hits DESC;
```

```
-- WINDOW FUNCTIONS & ADVANCED SQL
```

```
-- Top 3 Songs per Genre
```

```
SELECT *
FROM (
    SELECT
        track_genre,
        track_name,
        artists,
        popularity,
        ROW_NUMBER() OVER (PARTITION BY track_genre ORDER BY popularity DESC)
        AS rank_in_genre
    FROM spotify_staging
) ranked
WHERE rank_in_genre <= 3
ORDER BY track_genre, rank_in_genre;
```

```
-- Popularity Quartiles
```

```
SELECT
    track_name,
    artists,
    popularity,
    NTILE(4) OVER (ORDER BY popularity DESC) AS popularity_quartile
FROM spotify_staging
ORDER BY popularity DESC
```

```
LIMIT 20;
```

```
-- Artists Above Average Popularity
```

```
WITH artist_stats AS (  
    SELECT  
        artists,  
        ROUND(AVG(popularity), 2) AS avg_popularity,  
        COUNT(*) AS total_tracks  
    FROM spotify_staging  
    GROUP BY artists  
)  
SELECT *  
FROM artist_stats  
WHERE avg_popularity > (SELECT AVG(popularity) FROM spotify_staging)  
ORDER BY avg_popularity DESC;
```

```
-- DASHBOARD KPIs
```

```
SELECT  
    COUNT(*) AS total_tracks,  
    COUNT(DISTINCT artists) AS total_artists,  
    COUNT(DISTINCT album_name) AS total_albums,  
    COUNT(DISTINCT track_genre) AS total_genres,  
    ROUND(AVG(popularity), 2) AS avg_popularity,  
    ROUND(AVG(duration_ms) / 60000.0, 2) AS avg_duration_minutes,  
    ROUND(AVG(energy), 3) AS avg_energy,  
    ROUND(AVG(danceability), 3) AS avg_danceability  
FROM spotify_staging;
```

```
-- NORMALIZED SPOTIFY DATABASE SCHEMA
```

```
-- Users Table
```

```
CREATE TABLE users (  
    user_id SERIAL PRIMARY KEY,  
    username VARCHAR(50) NOT NULL UNIQUE,  
    email VARCHAR(100) NOT NULL UNIQUE,  
    password_hash TEXT NOT NULL,  
    full_name VARCHAR(100) NOT NULL,  
    country VARCHAR(50),  
    date_of_birth DATE,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
-- Artists Table
```

```
CREATE TABLE artists (  
    artist_id SERIAL PRIMARY KEY,  
    artist_name VARCHAR(150) NOT NULL,  
    country VARCHAR(50),  
    monthly_listeners BIGINT DEFAULT 0 CHECK (monthly_listeners >= 0),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Genres Table

```
CREATE TABLE genres (  
    genre_id SERIAL PRIMARY KEY,  
    genre_name VARCHAR(100) NOT NULL UNIQUE,  
    description TEXT  
);
```

-- Albums Table

```
CREATE TABLE albums (  
    album_id SERIAL PRIMARY KEY,  
    artist_id INTEGER NOT NULL,  
    album_name VARCHAR(200) NOT NULL,  
    album_type VARCHAR(30),  
    release_date DATE,  
    total_tracks INTEGER DEFAULT 0 CHECK (total_tracks >= 0),  
  
    CONSTRAINT fk_album_artist  
        FOREIGN KEY (artist_id)  
        REFERENCES artists(artist_id)  
        ON DELETE CASCADE  
);
```

-- Tracks Table

```
CREATE TABLE tracks (  
    track_id SERIAL PRIMARY KEY,  
    album_id INTEGER NOT NULL,  
    artist_id INTEGER NOT NULL,  
    genre_id INTEGER NOT NULL,  
  
    track_name VARCHAR(200) NOT NULL,  
    duration_ms INTEGER NOT NULL CHECK (duration_ms > 0),  
    popularity INTEGER CHECK (popularity BETWEEN 0 AND 100),  
    explicit BOOLEAN DEFAULT FALSE,  
    danceability NUMERIC(5,4) CHECK (danceability BETWEEN 0 AND 1),  
    energy NUMERIC(5,4) CHECK (energy BETWEEN 0 AND 1),  
    loudness NUMERIC(6,2),  
    speechiness NUMERIC(5,4) CHECK (speechiness BETWEEN 0 AND 1),  
    acousticness NUMERIC(6,5) CHECK (acousticness BETWEEN 0 AND 1),  
    instrumentalness NUMERIC(8,7) CHECK (instrumentalness BETWEEN 0 AND 1),  
    liveness NUMERIC(5,4) CHECK (liveness BETWEEN 0 AND 1),
```



```
valence NUMERIC(5,4) CHECK (valence BETWEEN 0 AND 1),
tempo NUMERIC(7,3),
time_signature INTEGER CHECK (time_signature BETWEEN 1 AND 7),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
CONSTRAINT fk_track_album
    FOREIGN KEY (album_id)
    REFERENCES albums(album_id),
```

```
CONSTRAINT fk_track_artist
    FOREIGN KEY (artist_id)
    REFERENCES artists(artist_id),
```

```
CONSTRAINT fk_track_genre
    FOREIGN KEY (genre_id)
    REFERENCES genres(genre_id)
```

```
);
```

```
-- Playlists Table
```

```
CREATE TABLE playlists (
    playlist_id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL,
    playlist_name VARCHAR(150) NOT NULL,
    description TEXT,
    is_public BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
CONSTRAINT fk_playlist_user
    FOREIGN KEY (user_id)
    REFERENCES users(user_id)
    ON DELETE CASCADE
```

```
);
```

```
-- Playlist Tracks Table
```

```
CREATE TABLE playlist_tracks (
    playlist_track_id SERIAL PRIMARY KEY,
    playlist_id INTEGER NOT NULL,
    track_id INTEGER NOT NULL,
    added_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
CONSTRAINT fk_playlist_tracks_playlist
    FOREIGN KEY (playlist_id)
    REFERENCES playlists(playlist_id)
    ON DELETE CASCADE,
```

```
CONSTRAINT fk_playlist_tracks_track
    FOREIGN KEY (track_id)
    REFERENCES tracks(track_id)
```

```

        ON DELETE CASCADE,

CONSTRAINT uq_playlist_track
    UNIQUE (playlist_id, track_id)
);

-- Listening History Table
CREATE TABLE listening_history (
    history_id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL,
    track_id INTEGER NOT NULL,
    played_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    play_duration_ms INTEGER CHECK (play_duration_ms >= 0),

CONSTRAINT fk_history_user
    FOREIGN KEY (user_id)
    REFERENCES users(user_id)
    ON DELETE CASCADE,

CONSTRAINT fk_history_track
    FOREIGN KEY (track_id)
    REFERENCES tracks(track_id)
    ON DELETE CASCADE
);

-- Subscription Plans Table
CREATE TABLE subscription_plans (
    plan_id SERIAL PRIMARY KEY,
    plan_name VARCHAR(50) NOT NULL UNIQUE,
    monthly_price NUMERIC(8,2) NOT NULL CHECK (monthly_price >= 0),
    audio_quality VARCHAR(30),
    offline_download BOOLEAN DEFAULT FALSE,
    ad_free BOOLEAN DEFAULT FALSE
);

-- Subscriptions Table
CREATE TABLE subscriptions (
    subscription_id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL,
    plan_id INTEGER NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE,
    status VARCHAR(20) NOT NULL CHECK (status IN ('Active','Expired','Cancelled')),

CONSTRAINT fk_subscription_user
    FOREIGN KEY (user_id)
    REFERENCES users(user_id)
    ON DELETE CASCADE,

```

```
CONSTRAINT fk_subscription_plan
    FOREIGN KEY (plan_id)
    REFERENCES subscription_plans(plan_id)
);
```

-- Recommendations Table

```
CREATE TABLE recommendations (
    recommendation_id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL,
    track_id INTEGER NOT NULL,
    recommendation_reason VARCHAR(200),
    recommended_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
CONSTRAINT fk_recommendation_user
    FOREIGN KEY (user_id)
    REFERENCES users(user_id)
    ON DELETE CASCADE,
```

```
CONSTRAINT fk_recommendation_track
    FOREIGN KEY (track_id)
    REFERENCES tracks(track_id)
    ON DELETE CASCADE
);
```

-- Performance Indexes

```
CREATE INDEX idx_users_email
ON users(email);
```

```
CREATE INDEX idx_artists_name
ON artists(artist_name);
```

```
CREATE INDEX idx_albums_artist
ON albums(artist_id);
```

```
CREATE INDEX idx_tracks_album
ON tracks(album_id);
```

```
CREATE INDEX idx_tracks_artist
ON tracks(artist_id);
```

```
CREATE INDEX idx_tracks_genre
ON tracks(genre_id);
```

```
CREATE INDEX idx_tracks_popularity
ON tracks(popularity DESC);
```

```
CREATE INDEX idx_playlists_user  
ON playlists(user_id);
```

```
CREATE INDEX idx_playlist_tracks_playlist  
ON playlist_tracks(playlist_id);
```

```
CREATE INDEX idx_playlist_tracks_track  
ON playlist_tracks(track_id);
```

```
CREATE INDEX idx_history_user  
ON listening_history(user_id);
```

```
CREATE INDEX idx_history_track  
ON listening_history(track_id);
```

```
CREATE INDEX idx_history_played_at  
ON listening_history(played_at);
```

```
CREATE INDEX idx_subscriptions_user  
ON subscriptions(user_id);
```

```
CREATE INDEX idx_subscriptions_plan  
ON subscriptions(plan_id);
```

```
CREATE INDEX idx_recommendations_user  
ON recommendations(user_id);
```

```
CREATE INDEX idx_recommendations_track  
ON recommendations(track_id);
```